

EE110B Lab#1

Nathan Taing

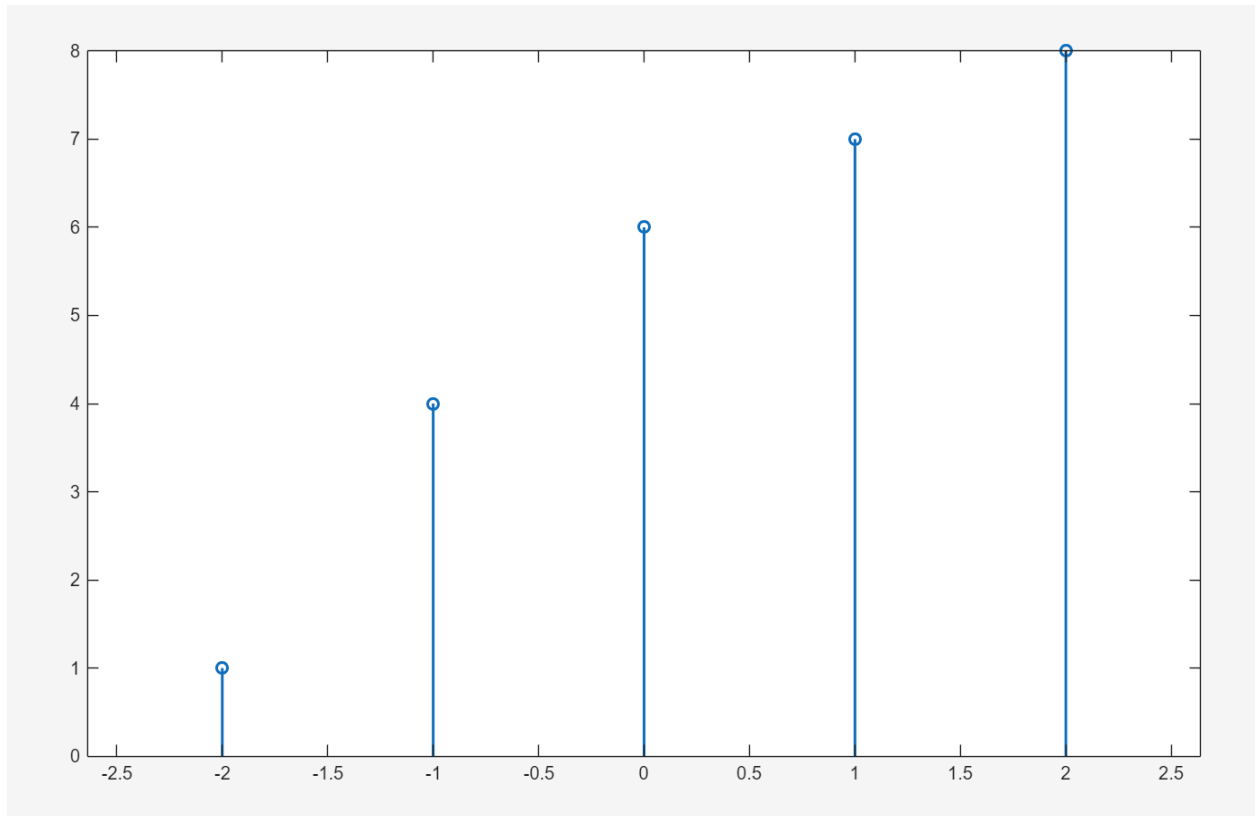
Basic MATLAB commands

1)

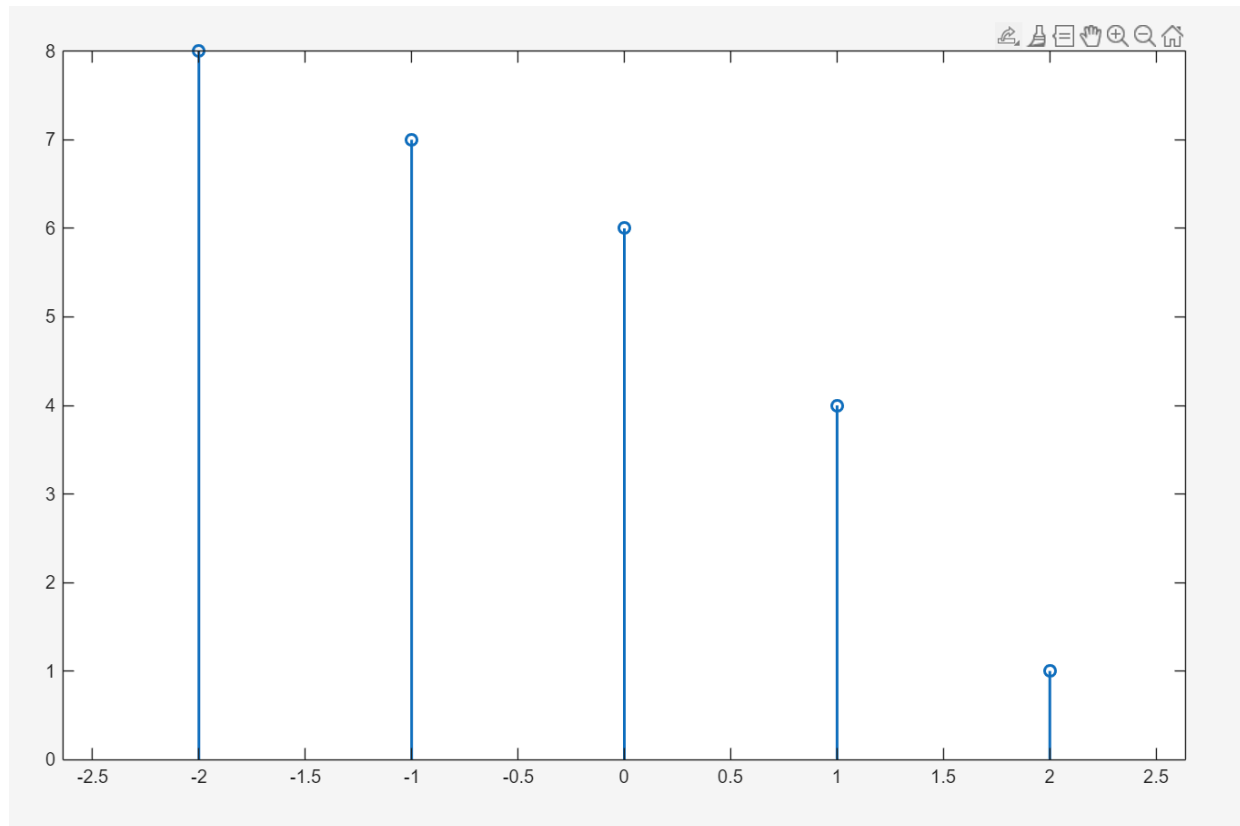
```
a = zeros(3, 4);
%creates a 3 x 4 matrix that is filled with zeros
b = ones(4, 2);
%creates a 4 x 2 matrix that is filled with ones
c = eye(3);
%creates a 3 x 3 identity matrix
d = size(a);
%returns the dimensions as a vector
abs([-3, 4]);
%returns the absolute value of each element
floor(1.2);
%rounds the element to the nearest integer less than or equal to
help ceil;
%provides definitions, instructions & examples of how to use the
function ceil
e = [1:2:10];
%creates vector starting at 1, incrementing by 2 and ends at or before
10
f = e';
%transposes the vector
g = e(2);
%grabs the element at row 2
h = sin(pi/2);
%sets h to sin(pi/2)
k = exp(1.0);
%sets k to e^1
ex1 = [1.1 2.2; 3.3 4.4];
%creates a 2 x 2 matrix with the top row having 1.1 and 2.2 and the
bottom row having 3.3 and 4.4
n = ex1(1,2);
%grabs the element at row 1, element 2
p = ex1(:, 2);
%grabs all rows at element 2
cmplx = 4 - 2i;
%creates a complex number
real(cmplx);
%provides the real component of cmplx
imag(cmplx);
%provides the imaginary component of cmplx
abs(cmplx);
%calculates the magnitude of cmplx
angle(cmplx);
%calculates the phase of cmplx
3^2;
%provides 3^2 (9)
3 == 3;
```

```
%determines if 3 is equal to 3 (true so returns 1)
3 == 2;
%determines if 3 is equal to 2 (false so returns 0)
```

2)

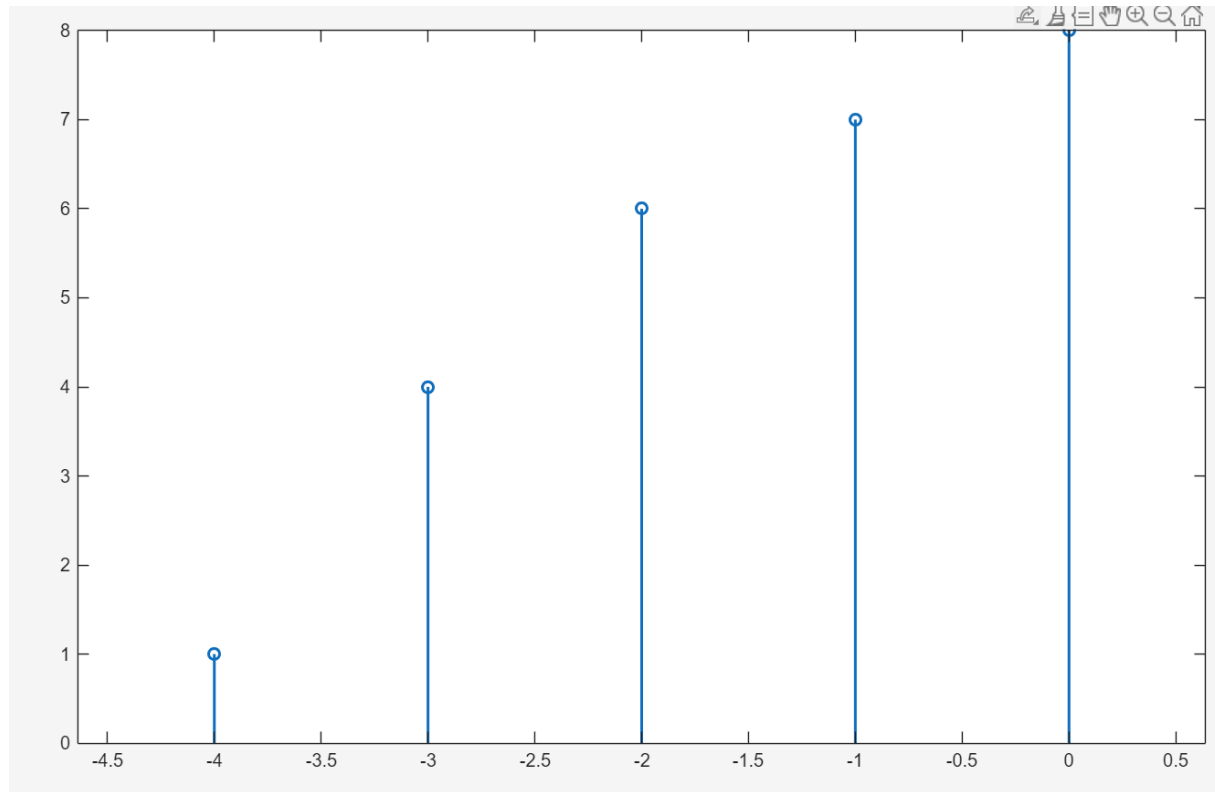


a)



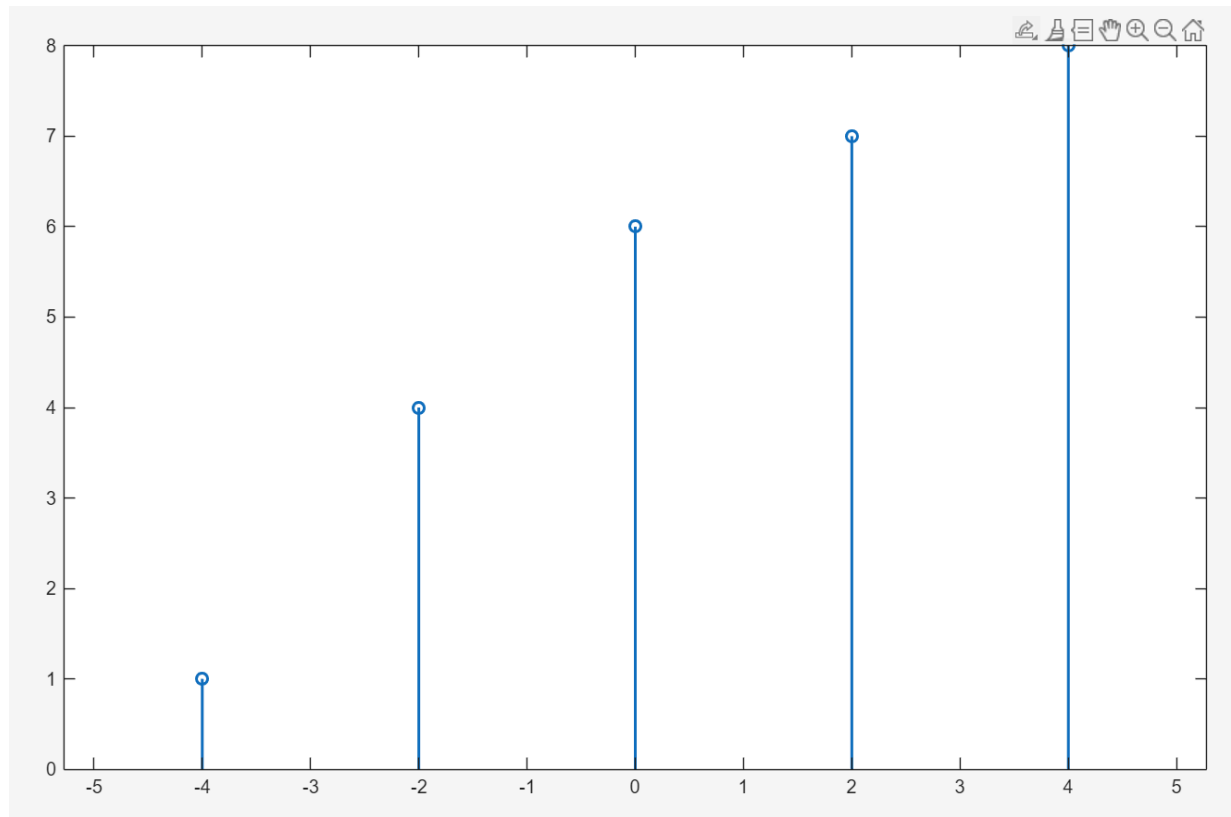
```
n = [-2:2];  
x = [1 4 6 7 8];  
n1 = n * -1;  
stem(n1, x, 'LineWidth', 1.5)
```

b)



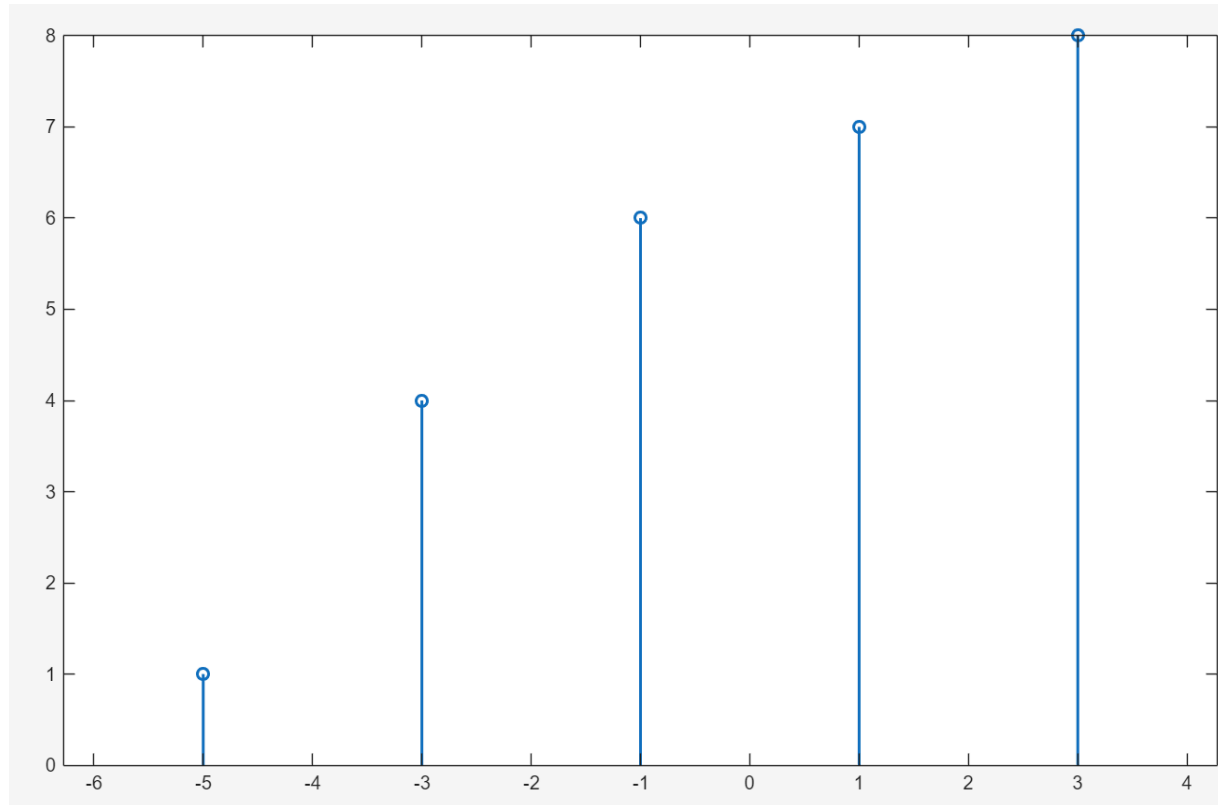
```
n = [-2:2];  
x = [1 4 6 7 8];  
n1 = n - 2;  
stem(n1, x, 'LineWidth', 1.5)
```

c)



```
n = [-2:2];  
x = [1 4 6 7 8];  
n1 = 2*n;  
stem(n1, x, 'LineWidth', 1.5)
```

d)



```
n = [-2:2];
x = [1 4 6 7 8];
n1 = 2*n - 1;
stem(n1, x, 'LineWidth', 1.5)
```

Plotting complex-valued signals

- See below
-

a)

$$x[n] = 2^{-n} e^{j\pi n/4}$$

$$= 2^{-n} \left(\cos\left(\frac{\pi n}{4}\right) - j \sin\left(\frac{\pi n}{4}\right) \right)$$

$\text{Re}(x[n]) = 2^{-n} \cos\left(\frac{\pi n}{4}\right)$

$\text{Im}(x[n]) = -2^{-n} \sin\left(\frac{\pi n}{4}\right)$

b)

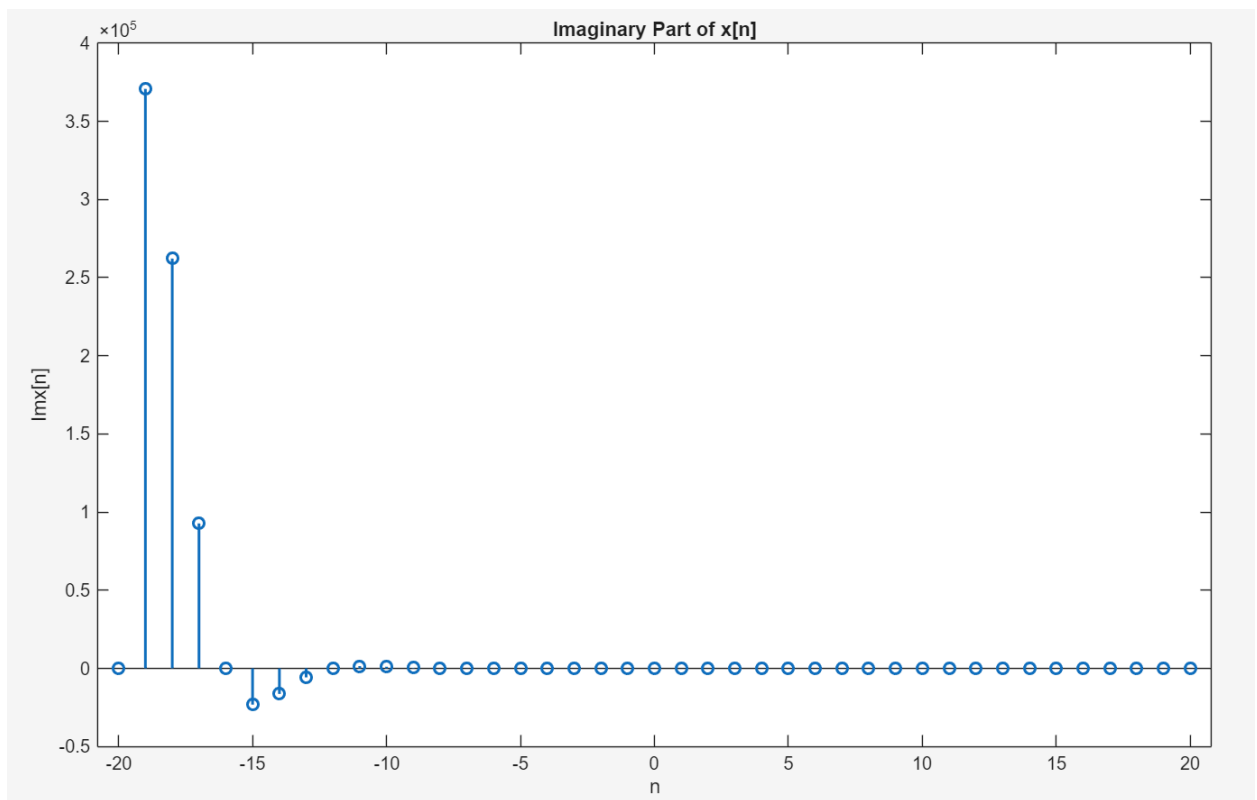
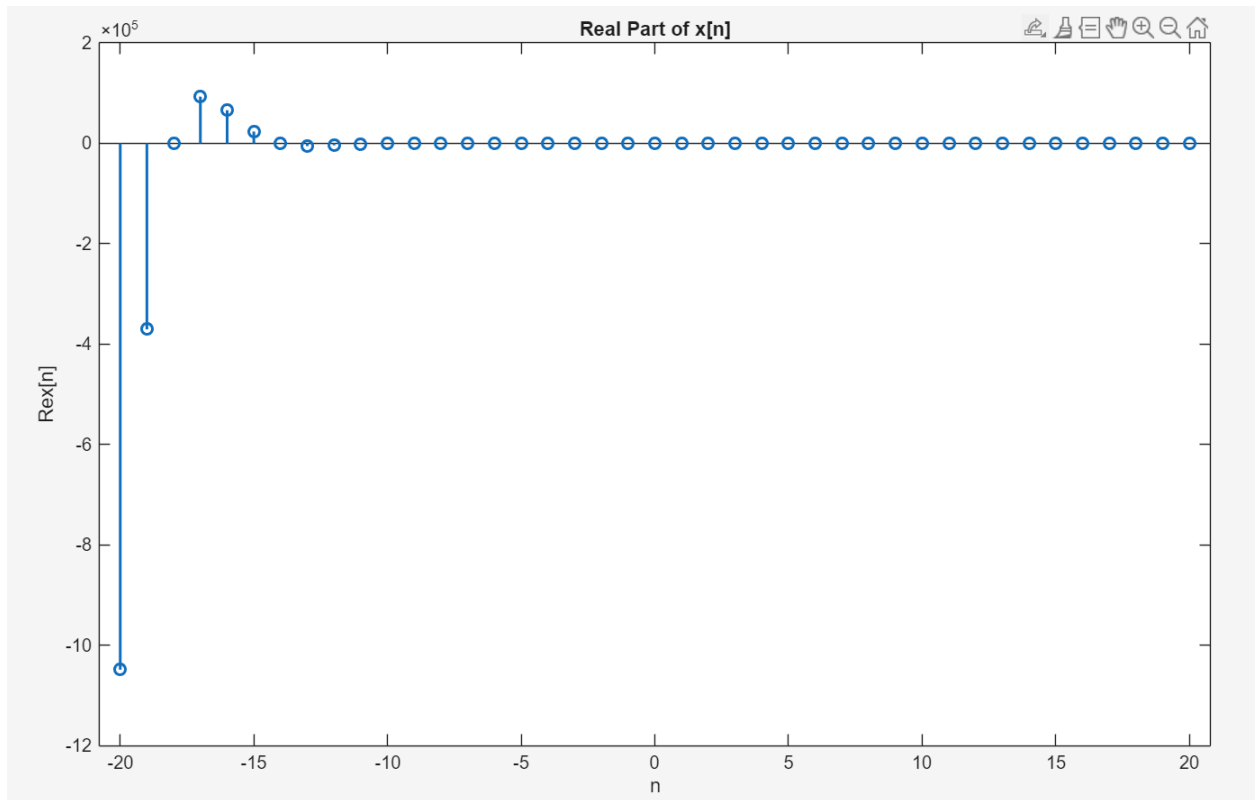
$$x[n] = 2^{-n} e^{j\pi n/4}$$

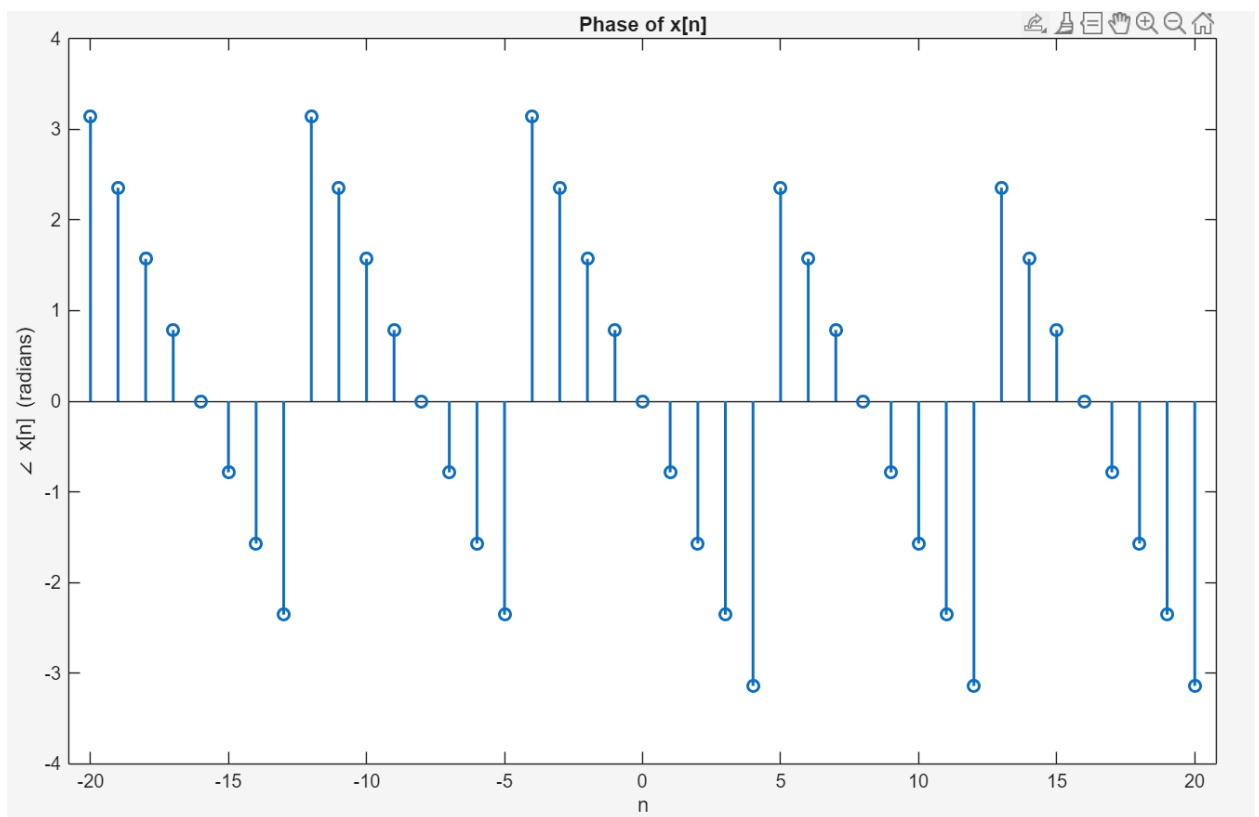
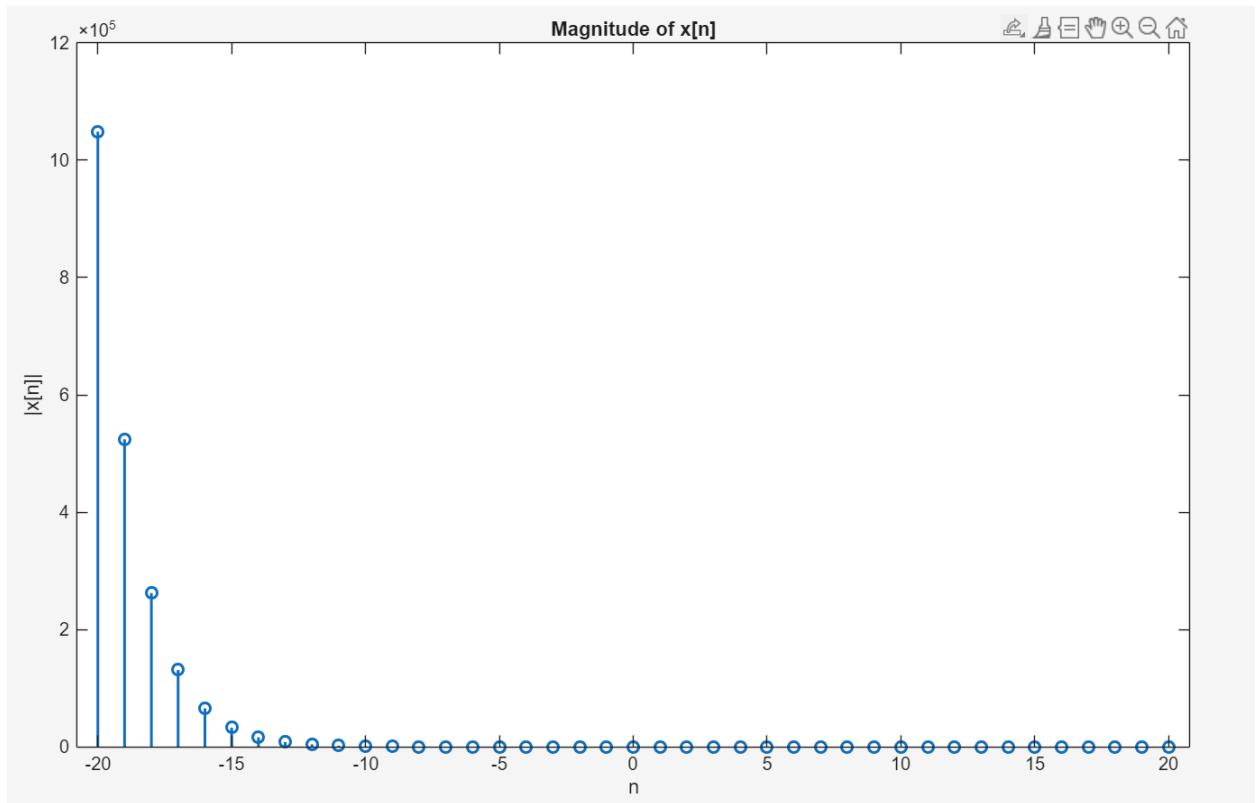
$|x[n]| = 2^{-n}$

$\angle x[n] = -\frac{\pi n}{4}$

Magnitude: 2^{-n}
 Phase: $-\frac{\pi n}{4}$

c)





```
n = -20:1:20;
x = (2.^(-n)).*(exp(-j*pi*n/4));
```

```

xr = real(x);
xi = imag(x);
xm = abs(x);
xp = angle(x);

figure                                     %plots real of x[n]
stem(n, xr, 'LineWidth', 1.5)
title('Real Part of x[n]')
xlabel('n')
ylabel('Re{x[n]}')

figure                                     %plots imaginary of x[n]
stem(n, xi, 'LineWidth', 1.5)
title('Imaginary Part of x[n]')
xlabel('n')
ylabel('Im{x[n]}')

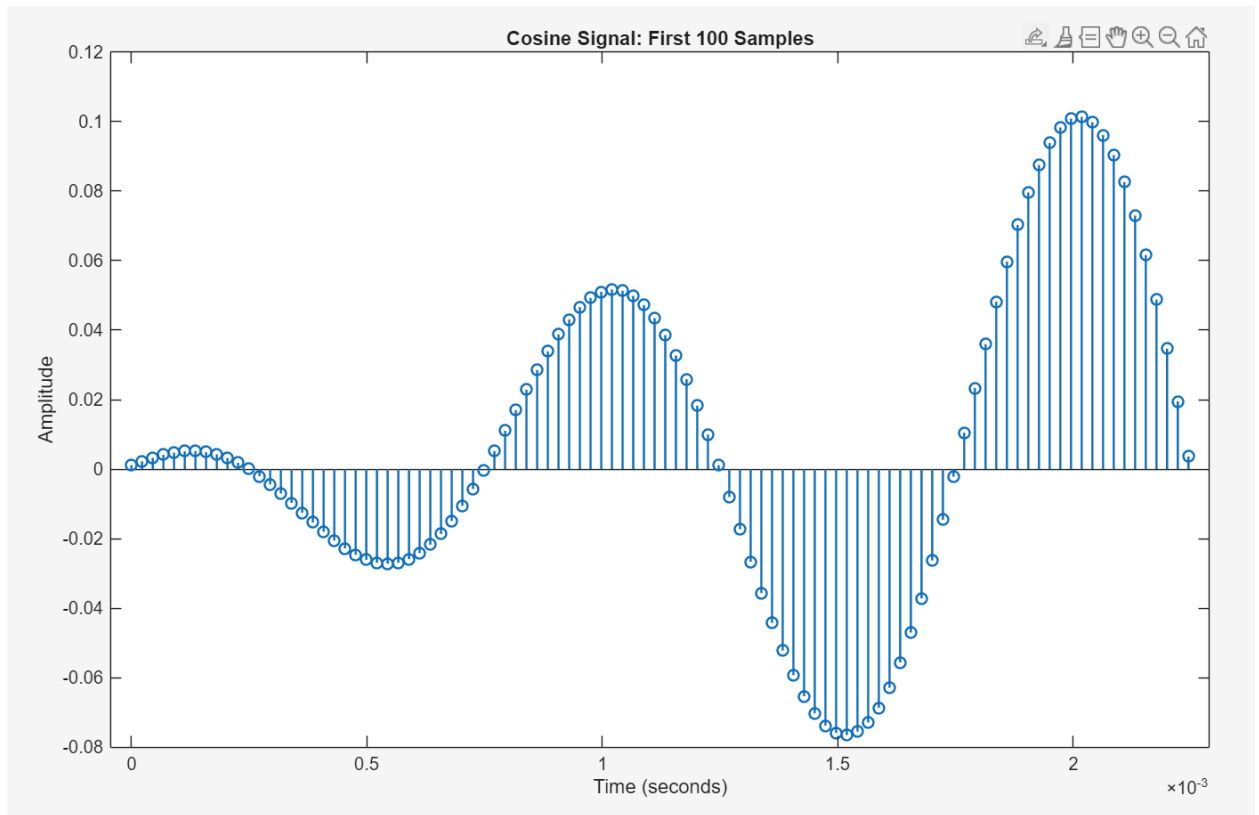
figure                                     %plots magnitude of x[n]
stem(n, xm, 'LineWidth', 1.5)
title('Magnitude of x[n]')
xlabel('n')
ylabel('|x[n]|')

figure                                     %plots phase of x[n]
stem(n, xp, 'LineWidth', 1.5)
title('Phase of x[n]')
xlabel('n')
ylabel('\angle x[n] (radians)')

```

Manipulation and perception of cosines and sines

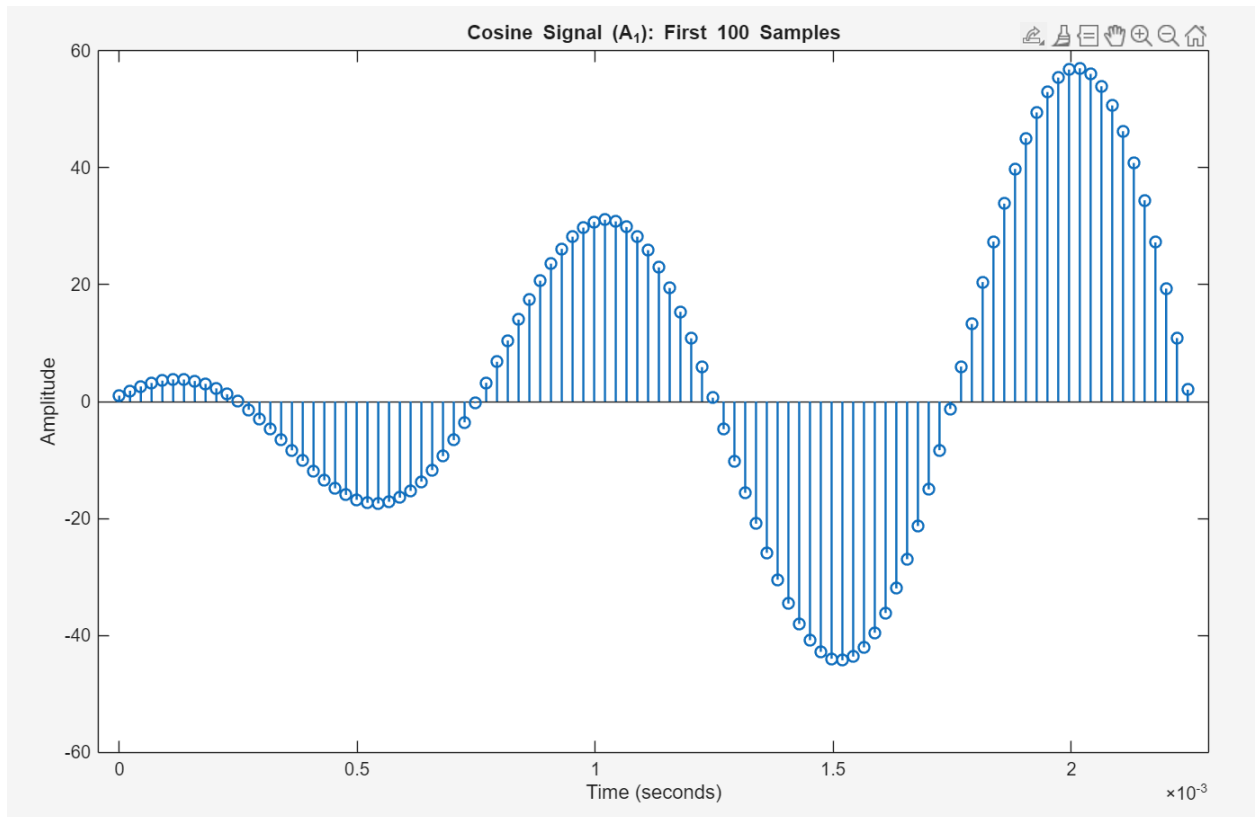
a)



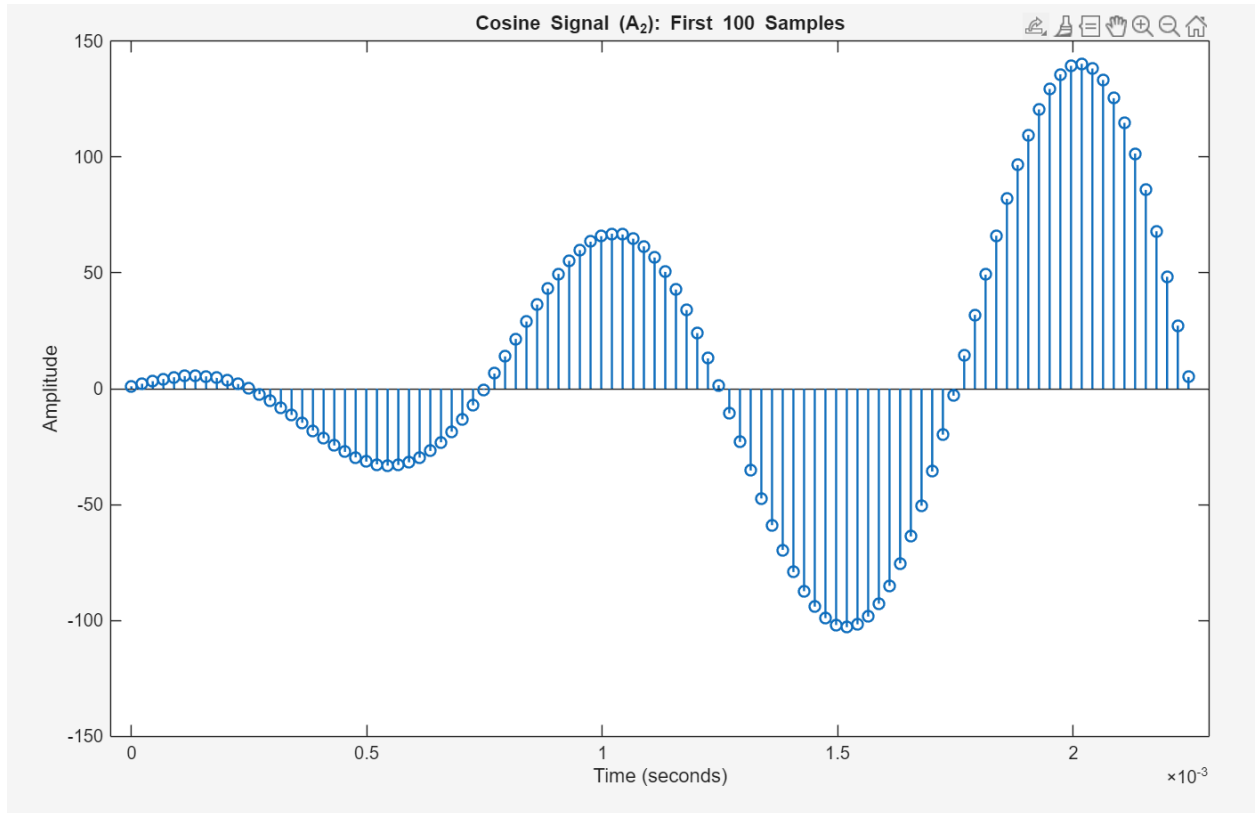
```
%plotting the cosine function
figure
stem(n(1:100), x(1:100), 'LineWidth', 1.2)
xlabel('Time (seconds)')
ylabel('Amplitude')
title('Cosine Signal: First 100 Samples')

T0 = 1/f0;
disp(num2str(T0)) (Period = 0.001s)
```

- b) A changes the volume of the sound that is produced. The amplitude is being modified, therefore affecting the perceived loudness of the sound.
- c)



```
% new plot for A_arr
figure
A_arr = (1:length(n)).^0.9;
x2 = A_arr.*cos((2*pi*f0*n)+phi);
stem(n(1:100), x2(1:100), 'LineWidth', 1.2)
xlabel('Time (seconds)')
ylabel('Amplitude')
title('Cosine Signal (A_1): First 100 Samples')
% playing
soundsc(x2,fs)
```



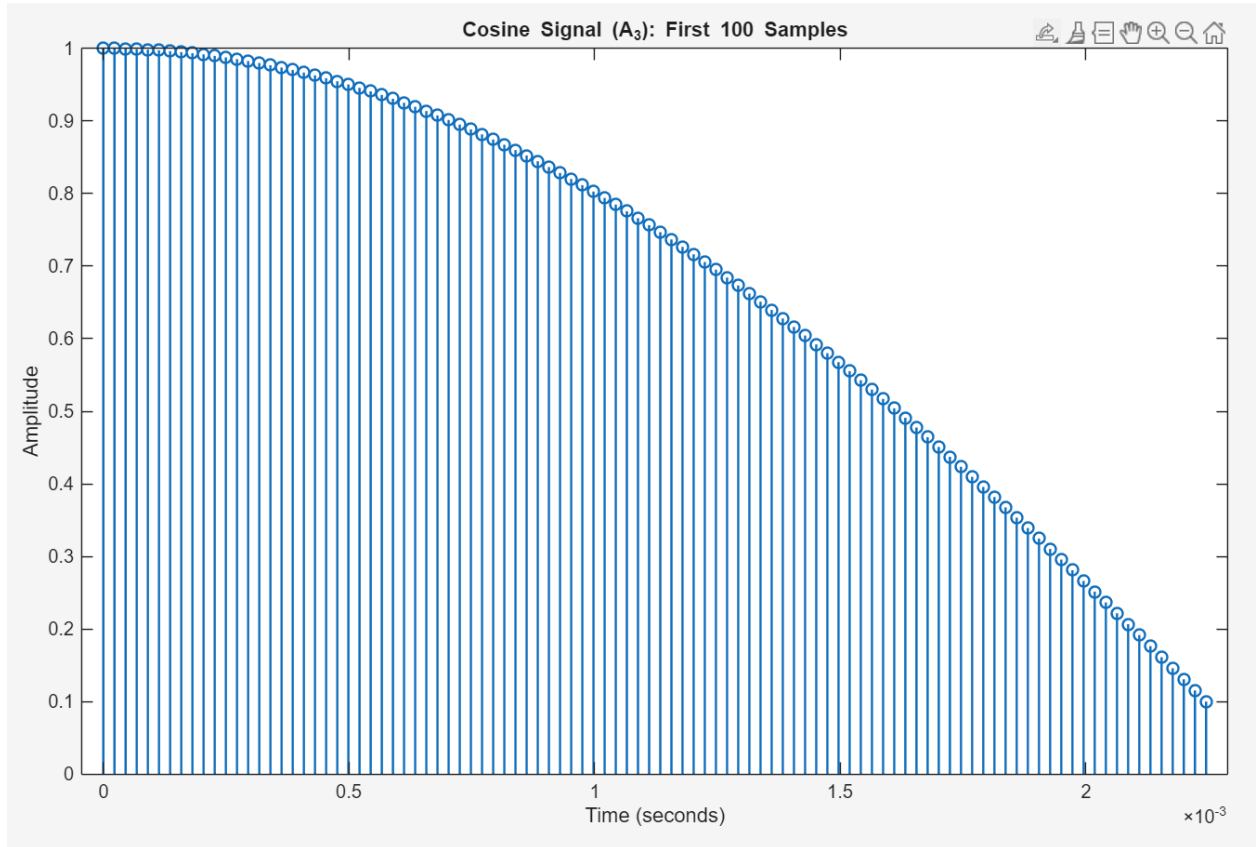
```
% new plot for second A_arr
figure
A_arr = (1:length(n)).^1.1;
x3 = A_arr.*cos((2*pi*f0*n)+phi);
stem(n(1:100), x3(1:100), 'LineWidth', 1.2)
xlabel('Time (seconds)')
ylabel('Amplitude')
title('Cosine Signal (A_2): First 100 Samples')
% playing
soundsc(x3,fs)
```

Both of the sounds start off quiet but grow to become louder. This is backed by the model as they both start off with low amplitudes and eventually grow louder. The first A_{arr} grows slowly from quiet to loud, however A_{arr2} grows loud significantly quicker.

- d) When changing f_0 , I was able to hear from the range 500 - 15000, anything outside that range I could not hear.
- e) For `soundsc(x(1:2:end), fs)`, the pitch became higher and the length was shortened compared to a `soundsc(x, fs)`. When doing `soundsc(x(1:4:end), fs)`, the pitch was even higher and shorter. This would make sense however because when modifying x , we are downsampling signals. This means that we are compressing the signal into a shorter time,

leading to a higher pitch (faster period) and shorter duration. For $x(1:4:\text{end})$ the same reasoning applies, however the downsampling has a greater factor.

f)



```
% new plot for f0_arr
f0_arr = linspace(100, 2000, length(n));
x3 = cos((2*pi*f0_arr.*n)+phi);
figure
stem(n(1:100), x3(1:100), 'LineWidth', 1.2)
xlabel('Time (seconds)')
ylabel('Amplitude')
title('Cosine Signal (A_3): First 100 Samples')
```

The signal, in this portion, is decreasing. However, if we were to sample the entire $x[n]$, we would see the shape of the signal contain cosine behavior. It is because we are modeling the first 100 samples that we see a decreasing signal.